

ปัญญาประดิษฐ์ยุคปัจจุบัน

สัปดาห์ที่ 12: เอเจนต์ AI และการใช้เครื่องมือ

ผศ. ดร. อธิพล ฟองแก้ว
ittipon@g.sut.ac.th

วันนี้

สัปดาห์ที่ 2 เรานิยาม "ตัวแทนที่มีเหตุผล" ว่าเป็นสิ่งที่รับรู้สภาพแวดล้อมแล้วลงมือทำ เพื่อให้ได้ผลลัพธ์ที่ดีที่สุด วันนี้เราจะสร้างมันด้วย LLM

ชั่วโมงที่ 1

เอเจนต์คืออะไร

- เวอร์กโฟลว์เทียบเอเจนต์
- โปรโตคอลเรียกเครื่องมือ
- ลูปของเอเจนต์
- เงื่อนไขหยุด

ชั่วโมงที่ 2

รูปแบบและหน่วยความจำ

- ReAct
- รูปแบบ 4 แบบ
- เอเจนต์หลายตัว
- หน่วยความจำ 4 ชนิด

ชั่วโมงที่ 3

ทำให้เชื่อถือได้

- โหมดล้มเหลว
- ชั้นควบคุม 5 ชั้น
- การประเมินเอเจนต์
- ลงมือทำในแล็บ

เชื่อมกลับสัปดาห์ที่ 2: เอเจนต์ LLM คือ agent function $f : \text{ประวัติการรับรู้} \rightarrow \text{การกระทำ}$ ที่เราไม่ได้เขียนกฎเอง แต่ให้โมเดลภาษาเป็นตัวคิด

ชั่วโมงที่ 1

เอเจนต์คืออะไร

นิยามเชิงปฏิบัติ

เอเจนต์คือ LLM ที่ทำงานในรูป โดยเลือกใช้เครื่องมือเองจนกว่างานจะเสร็จ

องค์ประกอบที่ขาดไม่ได้ 5 อย่าง

1. โมเดล ทำหน้าที่คิดและตัดสินใจ
2. เครื่องมือ ทำให้ลงมือทำได้จริง (MCP จากสัปดาห์ที่แล้ว)
3. ลูป ทำซ้ำจนกว่าจะบรรลุเป้าหมาย

1. หน่วยความจำ จำสิ่งที่ทำไปแล้ว
2. เงื่อนไขหยุด สำคัญที่สุดและถูกลืมบอยที่สุด

ระวิงคำโฆษณา: ระบบจำนวนมากที่เรียกตัวเองว่า "AI agent" จริง ๆ แล้วเป็น เวิร์กโฟลว์ ที่มีลำดับขั้นตอนตายตัว ความต่างคือ ใครเป็นคนตัดสินใจว่าจะทำอะไรต่อ ถ้าโปรแกรมเมอร์กำหนดไว้ล่วงหน้า นั่นคือเวิร์กโฟลว์ ถ้าโมเดลเลือกเอง นั่นคือเอเจนต์

เวิร์กโฟลว์กับเอเจนต์: เลือกอย่างไร

เวิร์กโฟลว์		เอเจนต์
ลำดับขั้น	กำหนดไว้ล่วงหน้า	โมเดลตัดสินใจตอนรัน
คาดเดาได้	สูง	ต่ำ
ต้นทุน	คงที่ ประเมินได้	แปรผัน อาจบานปลาย
ดีบักง่าย	ง่าย	ยาก
ทดสอบ	เขียน unit test ได้ตรง ๆ	ต้องมีชุดทดสอบเชิงพฤติกรรม
เหมาะกับ	งานที่รู้ขั้นตอนแน่นอน	งานที่ขั้นตอนขึ้นกับสิ่งที่เจอ

คำแนะนำที่ควรยึด: เริ่มจากสิ่งที่ง่ายที่สุดที่แก้ปัญหาได้
เรียก LLM ครั้งเดียวพอไหม → ถ้าไม่พอ ลองเวิร์กโฟลว์ → ถ้ายังไม่พอจึงใช้เอเจนต์
อย่าใช้เอเจนต์เพราะมันฟังดูทันสมัย

เราจะพูดถึงรูปแบบเวิร์กโฟลว์อย่างละเอียดในสัปดาห์หน้า

โปรโตคอลการเรียกเครื่องมือ

โมเดลไม่ได้รับโค้ดเอง มันแค่ขอให้เรารันแล้วส่งผลกลับไป

1. เราส่ง: ข้อความ + รายการเครื่องมือ (JSON Schema)
2. โมเดลตอบ: "ขอเรียก search_arxiv(query='attention', max=5)"
3. เรารันฟังก์ชันนั้นจริง ๆ ในโปรแกรมของเรา
4. เราส่งผลลัพธ์กลับเข้าไปเป็นข้อความบทบาท tool
5. โมเดลอ่านผล แล้วตอบ หรือขอเรียกเครื่องมืออีก

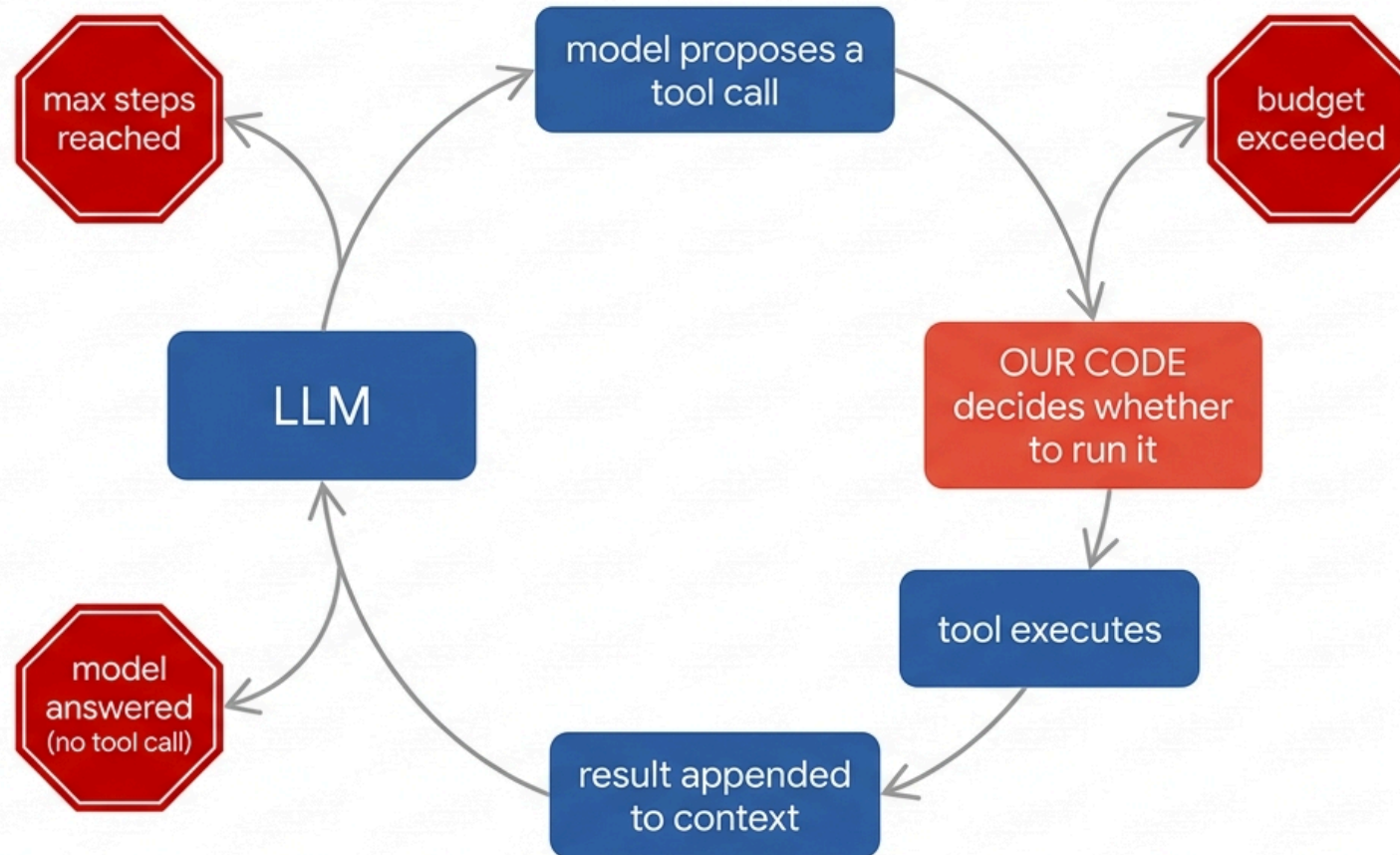
ประเด็นความปลอดภัยที่ต้องเข้าใจ: ขั้นที่ 3 คือโค้ดของเรา ที่ตัดสินใจว่าจะรันหรือไม่รัน โมเดลไม่มีอำนาจอะไรเลย นอกจากที่เราให้ ทุกการควบคุมเกิดขึ้นที่บรรทัดนี้

สิ่งที่ต้องเข้าใจให้ลึก: เมื่อโมเดล "ขอเรียกเครื่องมือ" มันแค่สร้างโทเคนที่มีรูปแบบพิเศษ ตามที่ถูกฝึกมา มันไม่ได้ "รู้" ว่าเครื่องมืออยู่จริงหรือไม่ นี่คือเหตุผลที่มันแต่งชื่อเครื่องมือขึ้นมาเองได้

นิยามเครื่องมือ

```
tools = [{  
    "type": "function",  
    "function": {  
        "name": "search_arxiv",  
        "description": "ค้นบทความวิชาการจาก arXiv ใช้เมื่อผู้ใช้งานถึงงานวิจัย "  
            "หรือต้องการอ้างอิงทางวิชาการ ห้ามใช้กับคำถามทั่วไป",  
        "parameters": {  
            "type": "object",  
            "properties": {  
                "query": {"type": "string", "description": "คำค้นภาษาอังกฤษ"},  
                "max_results": {"type": "integer", "default": 5, "maximum": 20},  
            },  
            "required": ["query"],  
        },  
    },  
},  
}]
```

description คือ **พรมบ่ต** ที่โมเดลใช้ตัดสินใจ ประโยค "ใช้เมื่อ..." และ "ห้ามใช้กับ..." มีผลต่อความแม่นยำมากกว่าการอธิบายว่าฟังก์ชันทำอะไร



the model runs nothing; our code does

รูปของเอเจนต์ เขียนเองได้ใน 25 บรรทัด

```
def run_agent(user_msg, tools, registry, max_steps=10, budget=0.50):
    messages = [{"role": "system", "content": SYSTEM_PROMPT},
                 {"role": "user", "content": user_msg}]
    spent = 0.0

    for step in range(max_steps):                                # เงื่อนไขหยุดที่ 1
        resp = client.chat.completions.create(
            model=MODEL, messages=messages, tools=tools)
        msg = resp.choices[0].message
        messages.append(msg)
        spent += estimate_cost(resp.usage)

        if not msg.tool_calls:                                    # เงื่อนไขหยุดที่ 2
            return msg.content                                    # ไม่เคลตอบแล้ว จบงาน
        if spent > budget:                                        # เงื่อนไขหยุดที่ 3
            return "หยุดเพราะเกินงบประมาณ"

        for call in msg.tool_calls:
            args = json.loads(call.function.arguments)
            result = registry[call.function.name](**args)        # รันจริงที่บรรทัดนี้
            messages.append({"role": "tool", "tool_call_id": call.id,
                             "content": str(result)[:4000]})    # ตัดผลไม่ให้ยาวเกิน

    return "หยุดเพราะครบจำนวนขั้นสูงสุด"
```

สังเกตเงื่อนไขหยุด 3 ชั้น เอเจนต์ที่ไม่มีเพดานจำนวนขั้นและงบประมาณ คือบักที่รอเกิด ไม่ใช่คุณสมบัติ

แบบฝึกหัด 1.1: ไล่อุปด้วยมือ

เอเจนต์มีเครื่องมือ `get_price(item)` และ `convert(amount, to)` งบประมาณ 0.10 USD ค่าเรียกโมเดลครั้งละ 0.03 USD เพดาน 5 ชั้น

ผู้ใช้งาน: "หนังสือเล่มนี้ราคาเท่าไรเป็นเงินบาท"

โมเดลตอบตามลำดับนี้

รอบ 1: ขอเรียก `get_price(item="หนังสือ")` -> ผลลัพธ์ "29.99 USD"
รอบ 2: ขอเรียก `convert(amount=29.99, to="THB")` -> ผลลัพธ์ "1,050 THB"
รอบ 3: ขอเรียก `convert(amount=29.99, to="THB")` -> ผลลัพธ์ "1,050 THB"
รอบ 4: ขอเรียก `convert(amount=29.99, to="THB")` -> ...

- ก) ลูปจบที่รอบไหน ด้วยเงื่อนไขข้อใด
- ข) เกิดอะไรขึ้นกับโมเดล และทำไม
- ค) เพิ่มการควบคุมอะไรเพื่อแก้ปัญหา

เฉลย 1.1

ก) จบที่รอบ 4 ด้วยเงื่อนไขงบประมาณ

รอบ ค่าใช้จ่ายสะสม เช็คงบ (> 0.10)

1	0.03	ไม่เกิน
2	0.06	ไม่เกิน
3	0.09	ไม่เกิน
4	0.12	เกิน หยุด

เพดาน 5 ขั้นยังไม่ถูกใช้เลย งบประมาณเป็นเงื่อนไขที่ทำงานก่อน

ข) โมเดลติดรูป เพราะเรียกเครื่องมือเดิมด้วยอาร์กิวเมนต์เดิมซ้ำ ๆ สาเหตุที่พบบ่อยคือ ผลลัพธ์ที่คืนมาไม่ได้ตอบคำถามในรูปแบบที่โมเดลคาดหวัง หรือพารามิเตอร์ระบบไม่ได้บอกว่า "เมื่อได้ข้อมูลครบแล้วให้ตอบ"

ค) การควบคุมที่ควรเพิ่ม

1. **ตรวจจบการเรียกซ้ำ** เกือบ (ชื่อเครื่องมือ, อาร์กิวเมนต์) ที่เคยเรียก ถ้าซ้ำให้คืนข้อความว่า "คุณเรียกสิ่งนี้ไปแล้ว ผลลัพธ์คือ ... ให้ตอบผู้ใช้ได้แล้ว"
2. เพิ่มในพารามิเตอร์ระบบว่า "เมื่อได้ข้อมูลครบให้ตอบทันที อย่าเรียกเครื่องมือซ้ำ"

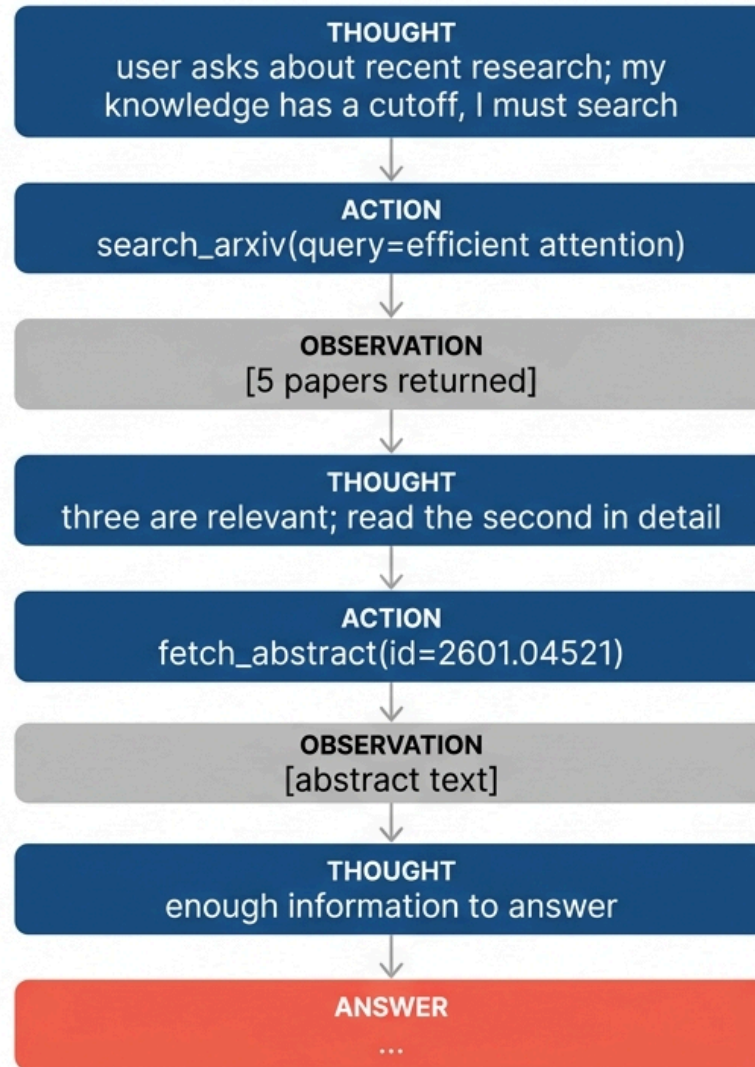
สรุปชั่วโมงที่ 1

- เอเจนต์ = โมเดล + เครื่องมือ + ลูป + หน่วยความจำ + **เงื่อนไขหยุด**
- ความต่างจากเวิร์กโฟลว์คือ **ใครตัดสินใจว่าจะทำอะไรต่อ**
- **เริ่มจากวิธีที่ง่ายที่สุดเสมอ** อย่าใช้เอเจนต์ถ้าเวิร์กโฟลว์พอ
- โมเดลไม่ได้รับอะไรเอง **โค้ดของเรา**เป็นผู้ตัดสินใจรัน จึงเป็นจุดควบคุมทุกอย่าง
- โมเดลแค่สร้างโทเคนที่มีรูปแบบพิเศษ มันจึงตั้งชื่อเครื่องมือขึ้นมาเองได้
- ต้องมีเงื่อนไขหยุดอย่างน้อย 3 ชั้น: จำนวนขั้น, โมเดลตอบแล้ว, งบประมาณ
- การติดลูปคือโหมดลัมเหลวที่พบบ่อยที่สุด ต้องตรวจจบการเรียกซ้ำ

พัก 10 นาที

ชั่วโมงที่ 2

รูปแบบและหน่วยความจำ



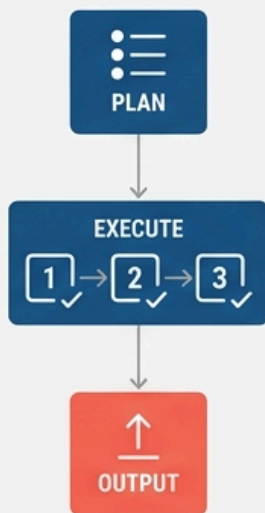
ReAct: ให้เหตุผลสลับกับลงมือทำ

ทำไมได้ผล: การเขียนเหตุผลออกมาก่อนเรียกเครื่องมือ ทำให้โมเดล

- เลือกเครื่องมือได้ตรงขึ้น (มี "พื้นที่คำนวณ" ตามที่คุณกันสัปดาห์ที่ 9)
- ตรวจจับได้ว่าผลลัพธ์ไม่สมเหตุสมผล
- และให้เรา **ดีบั๊กได้** เพราะเห็นว่ามันคิดผิดตรงไหน

ข้อสามสำคัญที่สุดในทางปฏิบัติ เมื่อเอเจนต์ทำงานผิด คำถามแรกคือ "มันคิดอะไรอยู่ตอนนั้น" ถ้าไม่มีร่องรอยความคิด คุณจะเห็นแค่ว่ามันเรียกเครื่องมือผิด แต่ไม่รู้ทำไม บันทึกร่องรอยทุกขั้นเสมอ แม้จะใช้โมเดลที่คิดเองได้แล้ว

1. Plan then execute



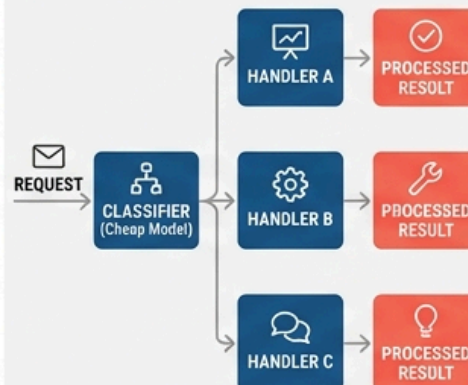
Good for: Sequential, deterministic tasks.

2. Reflection



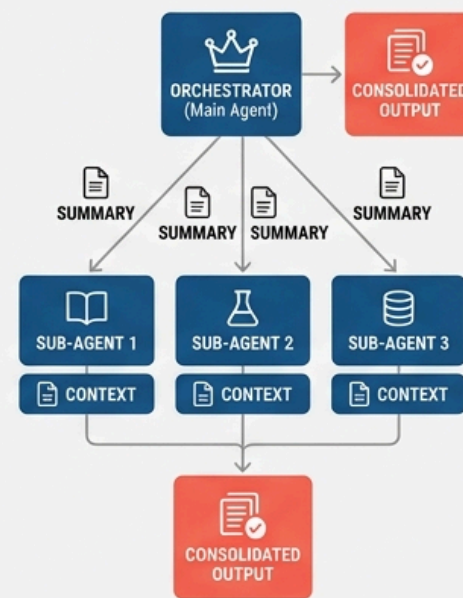
Good for: Iterative refinement and learning.

3. Router



Good for: Efficiently handling diverse query types.

4. Orchestrator and workers



Good for: Complex tasks requiring parallel sub-tasks.

รูปแบบและจุดที่ควรใช้

รูปแบบ	ได้อะไร	เสียอะไร	ใช้เมื่อ
วางแผนก่อนแล้วค่อยทำ	เห็นภาพรวม ตรวจสอบก่อนได้	แผนล้าสมัยเมื่อเจอสิ่งไม่คาดคิด	งานที่ขั้นตอนค่อนข้างแน่นอน
สะท้อนตัวเอง	คุณภาพสูงขึ้นชัดเจน	จ่ายค่าโทเคน 2 ถึง 3 เท่า	งานเขียน งานเขียนโค้ด
ตัวจัดเส้นทาง	ถูกและเร็วมาก	ต้องรู้ประเภทคำขอล่วงหน้า	ระบบจริงที่มีคำขอหลากหลาย
ผู้ประสานงานกับผู้ใช้ปฏิบัติ	ประหยัดบริบทของตัวหลัก	ประสานงานยาก แพงขึ้น	งานสำรวจที่ต้องอ่านข้อมูลมาก

เงื่อนไขที่ทำให้ "สะท้อนตัวเอง" ได้ผลจริง: ผู้วิจารณ์ต้องเก่งกว่าหรือมีข้อมูลมากกว่าผู้สร้าง
ถ้าให้โมเดลเดิมวิจารณ์งานตัวเอง ในสิ่งที่มันทำไม่ได้ตั้งแต่แรก มันจะวนอยู่กับที่และจ่ายเงินเพิ่มเปล่า ๆ
การใช้ผลจากเทสหรือ linter เป็นผู้วิจารณ์ ได้ผลดีกว่ามาก เพราะเป็นความจริงภายนอก

เอเจนต์หลายตัว: ใช้เมื่อไร

ได้ผลดีเมื่อ: งานย่อยเป็นอิสระต่อกัน และแต่ละงานต้องอ่านข้อมูลจำนวนมาก เอเจนต์ย่อยกลืนข้อมูลไว้ในบริบทตัวเอง ส่งกลับมาแค่ข้อสรุป

เอเจนต์หลัก: "หาว่าโค้ดเบสนี้จัดการ authentication อย่างไร"

--> ย่อย 1: อ่าน 30 ไฟล์ในโฟลเดอร์ auth	--> คืบสรุป 15 บรรทัด
--> ย่อย 2: อ่าน 20 ไฟล์ทดสอบ	--> คืบสรุป 10 บรรทัด
--> ย่อย 3: คืบเอกสารในเว็บ	--> คืบสรุป 8 บรรทัด

บริบทของเอเจนต์หลักโตขึ้นแค่ 33 บรรทัด แทนที่จะเป็น 50 ไฟล์

ไม่ได้ผลเมื่อ: งานต้องประสานกันแน่นแฟ้น เอเจนต์ย่อยมองไม่เห็นบริบทของกันและกัน จึงทำงานทับซ้อนหรือขัดแย้งกัน และค่าใช้จ่ายเพิ่มหลายเท่า

เอเจนต์เดี่ยวที่ออกแบบดี ชนะเอเจนต์หลายตัวที่ออกแบบลวก ๆ เกือบทุกครั้ง

หน่วยความจำ

ชนิด	เก็บที่ไหน	อายุ	ตัวอย่าง
บริบท	ในหน้าต่างบริบท	รอบสนทนา	ประวัติข้อความ
สถานะงาน	ไฟล์หรือตัวแปร	ตลอดงาน	รายการสิ่งที่ทำเสร็จแล้ว
ระยะยาว	ฐานข้อมูลเวกเตอร์	ข้ามการสนทนา	ความชอบของผู้ใช้ ข้อสรุปเดิม
เชิงกระบวนการ	ไฟล์คำสั่ง	ถาวร	วิธีทำงานที่เราสอนไว้ (สัปดาห์ที่ 13)

เทคนิคที่ใช้ได้จริงและง่ายที่สุด: ให้เอเจนต์เขียนบันทึกความคืบหน้าลงไฟล์ เช่น `progress.md` แล้วอ่านกลับตอนเริ่มรอบใหม่

การถ่ายสถานะออกนอกหน้าต่างบริบทแบบนี้ ทำให้เอเจนต์ทำงานยาว **ข้ามการบีบอัดบริบทได้** เพราะแม้บริบทจะถูกสรุปทับ ไฟล์ยังอยู่ครบ

แบบฝึกหัด 2.1: เลือกรูปแบบให้เหมาะกับงาน

จับคู่งานกับรูปแบบที่เหมาะสมที่สุด พร้อมเหตุผล

#

งาน

- ก. แชตบอตอบบริการลูกค้า รับคำถาม 5 ประเภท ตั้งแต่ง่ายมากถึงต้องส่งต่อพนักงาน
- ข. แพลเอกสารเทคนิค 200 หน้า ต้องได้คุณภาพสูง มีศัพท์เฉพาะเยอะ
- ค. ตอบคำถาม "ระบบนี้มีช่องโหว่ความปลอดภัยตรงไหนบ้าง" บนโค้ดเบส 2000 ไฟล์
- ง. สร้างรายงานประจำเดือนจากฐานข้อมูล ขั้นตอนเหมือนเดิมทุกเดือน

ตัวเลือก: ตัวจัดเส้นทาง · สะท้อนตัวเอง · ผู้ประสานงานกับผู้ปฏิบัติ · **ไม่ต้อง** ใช้เอเจนต์เลย

เฉลย 2.1

#	คำตอบ	เหตุผล
ก	ตัวจัดเส้นทาง	คำถามง่ายส่งโมเดลเล็ก (ถูกและเร็ว) คำถามยากส่งโมเดลใหญ่ ประเภทที่ต้องใช้คนก็ส่งต่อทันที ประหยัดได้มหาศาลในระบบจริง
ข	สะท้อนตัวเอง	มีเกณฑ์ชัด (ความหมายตกหล่นไหม ศัพท์ตรงไหม) และการวนซ้ำทำให้ดีขึ้นจริง คำนึงกับค่าโทเคนที่เพิ่ม
ค	ผู้ประสานงานกับผู้ใช้ปฏิบัติ	งานย่อยเป็นอิสระ (แต่ละโมดูลตรวจแยกกันได้) และแต่ละงานต้องอ่านไฟล์จำนวนมาก เข้าเงื่อนไขทั้งสองข้อพอดี
ง	ไม่ต้องใช้เอเจนต์	ขั้นตอนเหมือนเดิมทุกเดือน เขียนเป็นสคริปต์ธรรมดาหรือเวิร์กโฟลว์ตายตัว ถูกกว่า เร็วกว่า ดีบั๊กง่ายกว่า และคาดเดาได้

ข้อ ง คือบทเรียนที่สำคัญที่สุดของแบบฝึกหัดนี้ งานที่รู้ขั้นตอนแน่นอนไม่ควรใช้เอเจนต์ การให้โมเดลตัดสินใจในสิ่งที่เรารู้คำตอบอยู่แล้ว คือการจ่ายเงินเพื่อซื้อความไม่แน่นอน

สรุปชั่วโมงที่ 2

- **ReAct** ให้พื้นที่คำนวณและร่องรอยที่ดีบั๊กได้ **บันทึกร่องรอยเสมอ**
- รูปแบบ 4 แบบ แต่ละแบบมีเงื่อนไขที่ทำให้มันได้ผล ไม่ใช่ใช้ได้ทุกที่
- **สะท้อนตัวเองได้ผลก็ต่อเมื่อผู้วิจารณ์มีข้อมูลมากกว่าผู้สร้าง** เทสหรือ linter เป็นผู้วิจารณ์ที่ดีกว่า LLM ตัวเดิม
- **เอเจนต์หลายตัวช่วยเมื่องานย่อย เป็นอิสระ และ กินบริบทมาก**
- **หน่วยความจำมี 4 ชนิด และการเขียนสถานะลงไฟล์คือเทคนิคที่ง่ายและได้ผลที่สุด**
- **งานที่รู้ขั้นตอนแน่นอน ไม่ควร ใช้เอเจนต์**

พัก 10 นาที

ชั่วโมงที่ 3

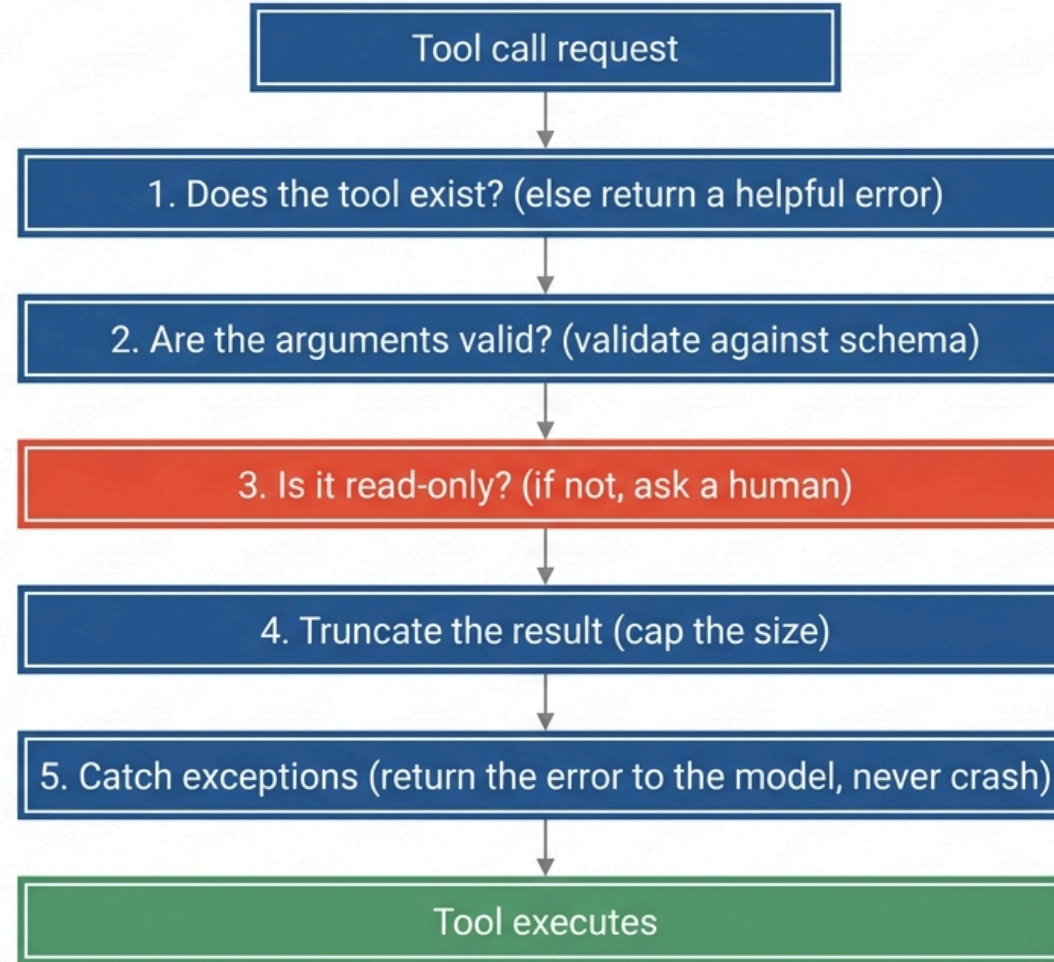
ทำให้เอเจนต์เชื่อถือได้

โหมดลุ่มเหลวที่พบบ่อย

อาการ	สาเหตุ	ทางแก้
วนลูปไม่จบ	เรียกเครื่องมือเดิมซ้ำ ๆ ด้วยอาร์กิวเมนต์เดิม	ตรวจจับการเรียกซ้ำ, เพดานจำนวนชั้น
เครื่องมือที่ไม่มีจริง	โมเดลแต่งชื่อฟังก์ชันขึ้นมา	ตรวจสอบชื่อกับ registry แล้วส่งข้อความผิดพลาดที่ช่วยแก้
บริบทล้น	ผลลัพธ์เครื่องมือยาวสะสมจนเต็ม	ตัดผลลัพธ์, สรุบทับ, ถ่ายลงไฟล์
ค่าใช้จ่ายบานปลาย	ไม่มีงบประมาณกำกับ	นับต้นทุนสะสมแล้วหยุดเมื่อเกิน
มันใจแต่ผิด	โมเดลรายงานความสำเร็จทั้งที่ไม่	ตรวจสอบผลด้วยโปรแกรม ไม่เชื่อคำรายงาน
ทำสิ่งที่ย้อนกลับไม่ได้	มีสิทธิ์เขียนหรือลบโดยไม่มีการอนุมัติ	แยกเครื่องมืออ่านกับเขียน, ให้คนอนุมัติ

ข้อที่ร้ายที่สุดคือ "มันใจแต่ผิด" เพราะระบบไม่แจ้งเตือนอะไรเลย เอเจนต์บอกว่า "แก้บักเรียบร้อยแล้วครับ" ทั้งที่ยังไม่ได้
รันเทส ให้ตรวจสอบผลลัพธ์ด้วยเงื่อนไขที่เขียนเป็นโปรแกรมได้เสมอ ไม่ใช่เชื่อคำรายงาน

Five control layers before a tool runs



การควบคุมที่ต้องมีในระบบจริง

```
READ_ONLY = {"search_arxiv", "read_file", "query_db"}

def execute(call, registry, auto_approve=True):
    name = call.function.name
    if name not in registry:                                # 1. ตรวจสอบว่ามีจริง
        return f"ไม่มีเครื่องมือชื่อ {name} เครื่องมือที่มีคือ {list(registry)}"

    args = validate(json.loads(call.function.arguments), SCHEMA[name]) # 2. ตรวจสอบอาร์กิวเมนต์

    if name not in READ_ONLY and not auto_approve:         # 3. ขออนุมัติสำหรับการเขียน
        if not ask_human(f"อนุญาตให้ {name}({args}) หรือไม่"):
            return "ผู้ใช้ปฏิเสธการดำเนินการนี้"

    try:
        return str(registry[name](**args))[:4000]          # 4. จำกัดขนาดผลลัพธ์
    except Exception as e:
        return f"เครื่องมือทำงานผิดพลาด: {e}"              # 5. ส่งข้อผิดพลาดกลับ ไม่ใช่ให้โปรแกรมตาย
```

ข้อ 5 สำคัญกว่าที่คิด: การส่งข้อความผิดพลาดกลับเข้าไปในบริบท ทำให้โมเดลแก้ตัวเองได้ เช่น เปลี่ยนอาร์กิวเมนต์ แล้วลองใหม่ ถ้าโปรแกรมพังไปเลย เอเจนต์จะไม่มีโอกาสนั้น นี่คือรูปแบบเดียวกับ **retry loop** ในสัปดาห์ที่ 9

มนุษย์อยู่ในวงจร (Human in the Loop)

ทำอัตโนมัติได้

- อ่านไฟล์ ค้นข้อมูล สอบถามฐานข้อมูล
- คำนวณ วิเคราะห์
- เขียนไฟล์ในพื้นที่ชั่วคราว
- รันเทสในสภาพแวดล้อมแยก

ต้องให้คนอนุมัติ

- ส่งอีเมลหรือข้อความ
- แก้หรือลบข้อมูลจริง
- ใช้จ่ายเงิน
- เผยแพร่สู่สาธารณะ
- เปลี่ยนการตั้งค่าระบบ

ยิ่งเอเจนต์เก่งขึ้น ยิ่งต้องออกแบบจุดอนุมัติ ให้ดี ไม่ใช่ลดลง เพราะเอเจนต์ที่เก่งจะทำงานที่ผิดได้เร็วกว่าและกว้างกว่า

แนวคิดสำคัญ: ให้เอเจนต์ทำงานใน **sandbox** ที่มันทำอะไรเสียหายถาวรไม่ได้ เช่น git branch แยก, container, หรือฐานข้อมูลสำเนา แล้วให้คนตรวจสอบก่อนรวมเข้าของจริง

การประเมินเอเจนต์

ต่างจากการประเมินพรมบัต เพราะเราสนใจเส้นทางไม่ใช่แค่คำตอบสุดท้าย

วัดผลลัพธ์

- อัตราความสำเร็จของงาน (วัดด้วยโปรแกรม)
- ความถูกต้องของคำตอบสุดท้าย

วัดกระบวนการ

- จำนวนขั้นที่ใช้
- ต้นทุนและเวลาต่องาน
- อัตราการเรียกเครื่องมือผิด
- อัตราการต้องให้คนเข้ามาช่วย

```
CASES = [  
    {"task": "หาบทความ 3 ชิ้นเรื่อง RAG ปี 2025 พร้อม DOI",  
     "check": lambda out, tr: len(extract_dois(out)) == 3 and all_valid(extract_dois(out))},  
]
```

เขียนฟังก์ชัน `check` ที่ตรวจได้ด้วยโปรแกรม ดีกว่าให้คนอ่านทีละอัน เพราะคุณจะรันมันซ้ำได้ทุกครั้งที่แก้พรมบัตหรือเปลี่ยนโมเดล

แบบฝึกหัด 3.1: เขียนฟังก์ชันตรวจสอบผล

เขียน `check(out, trace)` สำหรับงานต่อไปนี้ โดย ห้ามใช้ LLM ตัดสิน

#	งาน	ต้องตรวจอะไร
ก	"แก้บั๊กในไฟล์ <code>parser.py</code> ให้ทดสอบผ่าน"	?
ข	"หาคาหุน 3 ตัวนี้ แล้วบอกว่าตัวไหนแพงสุด"	?
ค	"สรุปเอกสารนี้ให้ไม่เกิน 100 คำ พร้อมอ้างอิงหน้า" ?	

คำใบ้: `trace` คือรายการขั้นตอนที่เอเจนต์ทำ แต่ละขั้นมี `tool, args, result`

เลข 3.1

```
# ก) ต้องรันเทสจริง ไม่ใช่แค่คำรายงานของเอเจนต์
def check_a(out, trace):
    r = subprocess.run(["pytest", "tests/test_parser.py", "-q"],
                       capture_output=True, text=True)
    return r.returncode == 0          # ความจริงภายนอก ไม่ใช่ข้อความจากโมเดล

# ข) ตรวจสอบผลลัพธ์และเส้นทาง
def check_b(out, trace):
    called = [t["args"]["symbol"] for t in trace if t.get("tool") == "get_price"]
    if set(called) != {"AAPL", "MSFT", "NVDA"}:
        return False                  # ต้องเรียกครบ ไม่ใช่แค่ราคาจากความจำ
    prices = {t["args"]["symbol"]: parse_price(t["result"]) for t in trace
              if t.get("tool") == "get_price"}
    return max(prices, key=prices.get) in out    # ตอบตัวที่แพงสุดจริง

# ค) ตรวจสอบข้อจำกัดที่วัดได้
def check_c(out, trace):
    return (len(out.split()) <= 100
            and re.search(r"\[หน้า \d+\]", out) is not None)
```

ข้อ ก คือหัวใจ เอเจนต์อาจตอบว่า "แก้แล้ว เทสผ่านหมด" ทั้งที่ไม่เคยรันเทสเลย การตรวจสอบต้องเป็น ความจริงภายนอก ที่เอเจนต์ควบคุมไม่ได้

ข้อ ข แสดงว่าเราต้องตรวจสอบ เส้นทาง ด้วย เพราะเอเจนต์อาจได้คำตอบถูก โดยบังเอิญ จากการเดาราคาจากความจำ ซึ่งจะพังทันทีเมื่อราคาเปลี่ยน

ช่วงลงมือทำ: labs/w12_agent.ipynb

แถบนี้เขียนลูปเอเจนต์เอง **ไม่ใช่เฟรมเวิร์ก** เพื่อให้เห็นว่าทุกอย่างเกิดขึ้นที่ไหน

1. ชั้นควบคุม 5 ชั้น

```
assert "ไม่มีเครื่องมือ" in execute("send_email", {...})
assert "อาร์กิวเมนต์ผิด" in execute("calculate", {"expr":
```

สังเกตว่า `execute` คืนสตริงเสมอ ไม่เคยโยน `exception`

2. เอเจนต์รอดจากการหลอน

`ScriptedLLM` จำลองการตั้งชื่อเครื่องมือ แล้วเราตรวจว่าเอเจนต์ได้ `error` กลับไปและทำงานต่อได้

3. เงื่อนไขหยุดทำงานจริง

```
out, spent, steps = run_agent("วนไปเรื่อย ๆ", Looper(), bu
assert "งบประมาณ" in out and steps < 8
```

เทียบกับที่เราคำนวณด้วยมือในแบบฝึกหัด 1.1

4. ชุดทดสอบพร้อม `check`

รัน `run_suite` แล้วดูตาราง `pass, steps, cost`

งานในคาบ: แก่ `ScriptedLLM` ให้จำลองพฤติกรรม "มั่นใจแต่ผิด" คือตอบว่าทำสำเร็จโดยไม่เคยเรียกเครื่องมือเลย แล้วเขียน `check` ที่จับได้

สรุป

- เอเจนต์ = โมเดล + เครื่องมือ + ลูป + หน่วยความจำ + **เงื่อนไขหยุด**
- **โค้ดของเรา**เป็นผู้ตัดสินใจรัน จึงเป็นจุดที่ควบคุมทุกอย่าง
- ทุกเอเจนต์ต้องมีเพดานจำนวนขั้น งบประมาณ และการตรวจสอบอาร์กิวเมนต์
- **ส่ง error กลับ** ให้โมเดลแก้ อย่า ให้โปรแกรมพัง
- แยกเครื่องมืออ่านออกจากเครื่องมือเขียน และให้มนุษย์อนุมัติสิ่งที่ย้อนกลับไม่ได้
- เอเจนต์หลายตัวช่วยได้เมื่องานย่อยเป็นอิสระและกินบริบทมาก
- **ประเมินด้วยความจริงภายนอก** ไม่ใช่คำรายงานของเอเจนต์ และตรวจทั้งผลลัพธ์และเส้นทาง

สัปดาห์หน้า: เราจะแกะดูว่าเครื่องมืออย่าง Claude Code, Gemini CLI และ Cursor สร้างเอเจนต์แบบนี้ขึ้นมาอย่างไร และเราจะขยายความสามารถมันด้วย Skills กับ Plugins ได้อย่างไร

การบ้านสัปดาห์ที่ 12

ส่วนที่ 1: ออกแบบ

1. ไล่รูปแบบฝึกหัด 1.1 ใหม่ โดยเปลี่ยนงบเป็น 0.20 USD และเพดานเป็น 3 ชั้น บอกว่าจบที่รอบไหนด้วยเงื่อนไขใด
2. เขียนฟังก์ชัน `check` สำหรับงาน "จองห้องประชุมให้ 5 คน วันพฤหัสบดี" โดยต้องตรวจทั้งผลลัพธ์และเส้นทาง

ส่วนที่ 2: แล็บ `labs/w12_agent.ipynb`

1. เพิ่มเครื่องมืออีก 2 ตัว โดยหนึ่งในนั้นต้อง **เขียนไฟล์** และอยู่นอก `READ_ONLY`
2. เขียนฟังก์ชัน `approve` ที่ถามผู้ใช้จริง ทดสอบทั้งกรณีอนุมัติและปฏิเสธ
3. เพิ่มการ **ตรวจจบการเรียกซ้ำ** ตามที่เฉลยในแบบฝึกหัด 1.1
4. ขยาย `TESTS` ให้ครบ 10 งาน แล้วเปรียบเทียบโมเดลจริง 2 ตัว รายงาน **อัตราสำเร็จ, ต้นทุน, จำนวนขั้นเฉลี่ย** พร้อมสรุปว่าควรเลือกตัวไหนเพราะอะไร

จบสัปดาห์ที่ 12